現在の配信を取得

DJ が今行っている配信の情報を取得します。タイトル・開始/終了時刻・リスナー数・挨拶・ カテゴリ・チャット凍結状態が一度に届きます。

メソッド・パス	`GET /v1/live`
必要な権限	`live.read`
成功レスポンス	`200`
配信中でないとき	`404`（`OAPI_MNGR_0301`）

```
export SPOON_BASE_URL="https://jp-openapi.spooncast.net"
export SPOON_ACCESS_TOKEN="{発行された access token}"
```

韓国リージョンなら `https://kr-openapi.spooncast.net` です — リージョン別アドレス。

リクエスト

```
GET /v1/live
```

ヘッダー

名前	必須	説明
`Authorization`	✔	`Bearer {access_token}` 形式です。

パスパラメータ・クエリパラメータ・リクエストボディは **ありません**。どの DJ の配信かは トークンが決めます — トークンひとつが同意した DJ ひとりなので、ボットが他人の配信を指す 場所がありません。

リクエスト例

```
curl -H "Authorization: Bearer $SPOON_ACCESS_TOKEN" "$SPOON_BASE_URL/v1/live"
```

レスポンス

```
{
  "liveId": 10001,
  "title": "今夜の深夜ラジオ",
  "startedAt": "2026-09-01T13:00:00Z",
  "closeAirTime": "2026-09-01T15:00:00Z",
  "listenerCount": 42,
  "totalListenerCount": 137,
  "welcomeMessage": "いらっしゃい！お便りはチャットへどうぞ",
  "isChatFrozen": false,
  "categories": ["talk"],
  "tags": ["深夜ラジオ", "お便り"]
}
```

フィールド	型	null	説明
`liveId`	number	いいえ	この配信回を指す id です。
`title`	string	いいえ	配信タイトルです。
`startedAt`	string	いいえ	開始時刻です。ISO-8601 UTC(`Z`)です。
`closeAirTime`	string	いいえ	終了 **予定** 時刻です。ISO-8601 UTC(`Z`)です。
`listenerCount`	number	いいえ	**今接続中の** リスナー数です。
`totalListenerCount`	number	いいえ	配信開始から今までに **訪れた累計** 人数です。

フィールド	型	null	説明
`welcomeMessage`	string	いいえ	DJ が部屋に掲げた挨拶です。書いていなければ空文字列です。
`isChatFrozen`	boolean	いいえ	チャットが凍結されているかを示します。
`categories`	string[]	いいえ	配信カテゴリです(`sing`・`talk` …)。
`tags`	string[]	いいえ	DJ が付けたハッシュタグです。

コード

```
// live.ts — node --experimental-strip-types live.ts

interface Live {
  liveId: number;
  title: string;
  startedAt: string;
  closeAirTime: string;
  listenerCount: number;
  totalListenerCount: number;
  welcomeMessage: string;
  isChatFrozen: boolean;
  categories: string[];
  tags: string[];
}

const res = await fetch(`${process.env.SPOON_BASE_URL}/v1/live`, {
  headers: { Authorization: `Bearer ${process.env.SPOON_ACCESS_TOKEN}` },
});

if (res.status === 404) {
  console.log('配信中ではありません。しばらくしてから確認します。');
} else if (!res.ok) {
  throw new Error(`取得に失敗しました: ${res.status}`);
} else {
  const live = (await res.json()) as Live;
```

```
  console.log(`[${live.liveId}] ${live.title} – リスナー ${live.listenerCount}人 (累計 ${live.totalListenerCount}人)`);
  if (live.isChatFrozen) console.log('チャットが凍結されています。今は発言しません。');
}
```

```
node --experimental-strip-types live.ts
```

注意

`isChatFrozen` が `true` **のときはチャットを送れません。**チャットを送る が 失敗します。ボットもリスナーと同じルールに従うためです。発言する前にこの値を見て止めれば、失敗を先に避けられます。

`liveId` はボットが **配信回を区別する値** です。「今回の配信で 1 回だけ」といったルールの 冪等キーとして使い、再接続したときに同じ配信か新しい配信かを判断するのにも使います。`startedAt` で代用すると、値が少し揺れただけでひとつの配信が 2 つに分かれます。

`listenerCount` と `totalListenerCount` を混同しないでください。前者は **今接続中**、後者は **訪れた累計** です。累計は減りません。

配信が始まったかを確認するためにこのエンドポイントをポーリングする必要はありません。イベントストリームは配信中でなければ接続前に `404` を返すので、接続を再試行するだけで足ります — 接続ループ を参照してください。

エラー

HTTP	`detailCode`	何が起きたか	どうするか
`401`	なし	トークンがないか無効です。	DJ に再連携を案内します。
`403`	なし	`live.read` の権限がありません。	権限を追加して再同意を得ます。
`404`	`OAPI_MNGR_0301`	配信中ではありません。	エラーではなく状態です。しばらくして呼び直します。
`502`	`OAPI_MNGR_2302`	配信サーバーのエラーです。	バックオフしてから再試行します。

この表にない状態も返ることがあります。 誤ったメソッド・`Content-Type`(`400`・`405`・`415`、`OAPI_MNGR_0103`)・呼び出し上限(`429`、ボディなし)・サーバーエラー(`500`、`OAPI_MNGR_1002`)は **すべてのエンドポイントに共通** です — エラーコード にあります。

関連ドキュメント

- エンドポイント一覧 — エンドポイントの全一覧です。
- リスナー一覧を取得 — 今部屋に誰がいるかを確認します。
- イベントストリーム — 配信中の出来事をリアルタイムに受け取ります。
- 権限を選ぶ — `live.read` が何を与えるかです。
- エラーコード — エラーコードの一覧です。